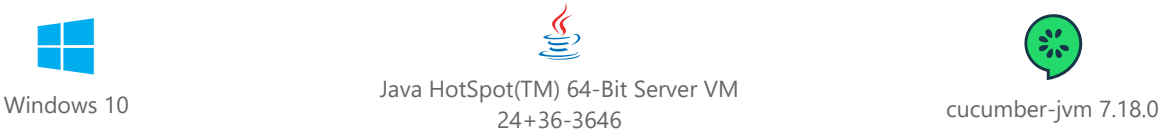


100% passed
85 executed

1 minute ago
last run

2.85 seconds
duration



You can search with plain text or [Cucumber Tag Expressions](#) to filter the output

✓ classpath:features/amount_to_words.feature

Feature: Amount to Words Conversion

As a payroll system
I want to convert a monetary amount into English words
So that the Payslip and Cheque show the correct "Pay to the order of ... Dollars" text

Scenario: Zero amount converts to "Zero Dollars"

- ✓ When I convert the amount 0 to words
- ✓ Then the result should be "Zero Dollars"

Scenario: Whole dollar amount converts without cents

- ✓ When I convert the amount 100 to words
- ✓ Then the result should be "One Hundred Dollars"

Scenario: Amount with cents includes "and ... Cents"

- ✓ When I convert the amount 1234.56 to words
- ✓ Then the result should be "One Thousand Two Hundred Thirty Four Dollars and Fifty Six Cents"

Scenario Outline: Hundred, Thousand, Million and Billion boundaries

- ✓ When I convert the amount <amount> to words
- ✓ Then the result should be "<words>"

Examples:

	amount	words
✓	100	One Hundred Dollars
✓	1000	One Thousand Dollars
✓	1000000	One Million Dollars
✓	1000000000	One Billion Dollars

Scenario: Negative amount throws an exception

- ✓ **When** I convert the amount -50 to words
- ✓ **Then** an amount conversion IllegalArgumentException should be thrown

Scenario Outline: Two-digit numbers convert correctly

- ✓ **When** I convert the amount <amount> to words
- ✓ **Then** the result should be "<words>"

Examples:

	amount	words
✓	42	Forty Two Dollars
✓	19	Nineteen Dollars

Scenario: Rounding edge case rounds cents up to the next dollar

- ✓ **When** I convert the amount 12.999 to words
- ✓ **Then** the result should be "Thirteen Dollars"

✓ ✓ classpath:features/db_connection.feature

Feature: Database Connectivity

As the payroll system

I want to establish a connection to the MySQL database

So that employee, payroll and payslip data can be read and written

@db

Scenario: Application can connect to the configured MySQL database

- ✓ **When** I try to open a database connection
- ✓ **Then** the connection should be successful and open

✓ ✓ classpath:features/employee_form_validation.feature

Feature: Employee Form Validation

As the payroll system

I want to validate names, phone numbers, bank account numbers and dates entered on the Add/Update Employee screens

So that only well-formed data reaches the database

Scenario Outline: Validating employee names

- ✓ **When** I validate the name "<name>"
- ✓ **Then** the name should be considered <validity>

Examples:

	name	validity
✓	John	valid
✓	Anne-Marie	valid
✓	O'Brien	valid
✓	Mary Jane	valid
✓		invalid
✓	123John	invalid
✓	John123	invalid
✓	John_Doe	invalid
✓	John@Doe	invalid

Scenario Outline: Validating contact numbers

- ✓ **When** I validate the contact number "<phone>"
- ✓ **Then** the contact number should be considered <validity>

Examples:

	phone	validity
✓	01712345678	valid
✓	+8801712345678	valid
✓	017-1234-5678	valid
✓	1234567	valid
✓		invalid
✓	123	invalid
✓	12345abc	invalid

Scenario Outline: Validating bank account numbers

- ✓ **When** I validate the account number "<accountNo>"
- ✓ **Then** the account number should be considered <validity>

Examples:

	accountNo	validity
✓	1234	valid
✓	GB29-NWBK-6016-1331-9268-19	valid
✓	ACC1234567890	valid
✓		invalid
✓	123	invalid

	accountNo	validity
✓	abc@123	invalid

Scenario Outline: Converting input dates to SQL date format

- ✓ **When** I convert the input date "<inputDate>" to SQL format
- ✓ **Then** the SQL date should be "<sqlDate>"

Examples:

	inputDate	sqlDate
✓	11/15/2025	2025-11-15
✓	01/01/2000	2000-01-01
✓	12/31/1999	1999-12-31

Scenario Outline: Converting an invalid input date raises an error

- ✓ **When** I convert the input date "<inputDate>" to SQL format
- ✓ **Then** a date conversion error should occur

Examples:

	inputDate
✓	2025-11-15
✓	13/01/2025
✓	11/31/2025
✓	not-a-date

✓ ✓ classpath:features/employees_model.feature

Feature: Employees Data Model

As the payroll system

I want the Employees model to correctly store and retrieve employee data

So that every screen (Add, Update, View, Search, Payslip) works with consistent data

Scenario: Setting and getting all employee fields

- ✓ **Given** a new employee record
- ✓ **When** I set the employee id to 101
- ✓ **And** I set the first name to "John"
- ✓ **And** I set the last name to "Doe"
- ✓ **And** I set the designation to "Software Engineer"
- ✓ **And** I set the contact number to "01711111111"
- ✓ **And** I set the email id to "john.doe@example.com"
- ✓ **And** I set the department to "Engineering"
- ✓ **And** I set the salary to 50000.0

- ✓ **Then** the employee id should be 101
- ✓ **And** the first name should be "John"
- ✓ **And** the last name should be "Doe"
- ✓ **And** the designation should be "Software Engineer"
- ✓ **And** the contact number should be "01711111111"
- ✓ **And** the email id should be "john.doe@example.com"
- ✓ **And** the department should be "Engineering"
- ✓ **And** the salary should be 50000.0

Scenario: A newly created employee record has default values

- ✓ **Given** a new employee record
- ✓ **Then** the employee id should be 0
- ✓ **And** the salary should be 0.0
- ✓ **And** the first name should be null

Scenario Outline: Updating the salary field

- ✓ **Given** a new employee record
- ✓ **When** I set the salary to <salary>
- ✓ **Then** the salary should be <salary>

Examples:

	salary
✓	0.0
✓	1000.0
✓	99999.99

✓ ✓ classpath:features/login_auth.feature

Feature: Login Authentication

As the payroll system
I want to validate username/password on the Login tab
So that only authorized users can access the system

Scenario Outline: Login attempts

- ✓ **When** I attempt to login with username "<username>" and password "<password>"
- ✓ **Then** the login should be <result>

Examples:

	username	password	result
✓	admin	admin123	successful
✓	admin	wrongpass	unsuccessful
✓	wronguser	admin123	unsuccessful

	username	password	result
✓	wronguser	wrongpass	unsuccessful
✓	Admin	admin123	unsuccessful
✓			unsuccessful

✓ classpath:features/payroll_calculator.feature

Feature: Payroll Calculation

As a payroll system

I want to calculate gross salary, deductions and net salary correctly

So that employee pay is accurate and consistent across Add/Update/Payslip screens

Background:

- ✓ **Given** a pay rate of 10.0 per hour

Scenario: Normal hours under 40 produce only normal salary

- ✓ **Given** the total hours worked is 35
- ✓ **When** I calculate the payroll
- ✓ **Then** the normal salary should be 350.0
- ✓ **And** the gross salary should be 350.0

Scenario: Exactly 40 hours is the boundary for normal salary

- ✓ **Given** a pay rate of 15.0 per hour
- ✓ **And** the total hours worked is 40
- ✓ **When** I calculate the payroll
- ✓ **Then** the normal salary should be 600.0
- ✓ **And** the overtime salary should be 0.0

Scenario: Total hours above 40 are capped at 40 for normal salary

- ✓ **Given** the total hours worked is 50
- ✓ **When** I calculate the payroll
- ✓ **Then** the normal salary should be 400.0

Scenario: Overtime hours are paid at 1.5x the pay rate

- ✓ **Given** the total hours worked is 40
- ✓ **And** overtime hours of 5
- ✓ **When** I calculate the payroll
- ✓ **Then** the overtime salary should be 75.0
- ✓ **And** the gross salary should be 475.0

Scenario: Doubletime hours are paid at 2x the pay rate

- ✓ **Given** the total hours worked is 40
- ✓ **And** doubletime hours of 4
- ✓ **When** I calculate the payroll

- ✓ **Then** the doubletime salary should be 80.0
- ✓ **And** the gross salary should be 480.0

Scenario: Extended, holiday paid and special work hours use the normal pay rate

- ✓ **Given** extended hours of 2
- ✓ **And** holiday paid hours of 3
- ✓ **And** special work hours of 4
- ✓ **When** I calculate the payroll
- ✓ **Then** the extended salary should be 20.0
- ✓ **And** the holiday paid salary should be 30.0
- ✓ **And** the special work salary should be 40.0
- ✓ **And** the gross salary should be 90.0

Scenario: Bonus amount is added to gross salary and is taxed

- ✓ **Given** the total hours worked is 40
- ✓ **And** a bonus amount of 100
- ✓ **When** I calculate the payroll
- ✓ **Then** the gross salary should be 500.0
- ✓ **And** the tax deduction should be 100.0
- ✓ **And** the medical insurance should be 50.0
- ✓ **And** the net salary should be 350.0

Scenario: All zero inputs result in all zero outputs

- ✓ **Given** the total hours worked is 0
- ✓ **When** I calculate the payroll
- ✓ **Then** the gross salary should be 0.0
- ✓ **And** the tax deduction should be 0.0
- ✓ **And** the medical insurance should be 0.0
- ✓ **And** the net salary should be 0.0

Scenario Outline: Negative inputs are rejected

- ✓ **Given** a pay rate of <payRate>
- ✓ **And** the total hours worked is <totalHours>
- ✓ **And** overtime hours of <overtimeHours>
- ✓ **And** doubletime hours of <doubletimeHours>
- ✓ **And** extended hours of <extendedHours>
- ✓ **And** holiday paid hours of <holidayHours>
- ✓ **And** special work hours of <specialHours>
- ✓ **And** a bonus amount of <bonus>
- ✓ **When** I calculate the payroll
- ✓ **Then** a payroll IllegalArgumentException should be thrown

Examples:

	payRate	totalHours	overtimeHours	doubletimeHours	extendedHours	holidayHours	specialHours	bonus
✓	-1	0	0	0	0	0	0	0

	payRate	totalHours	overtimeHours	doubletimeHours	extendedHours	holidayHours	specialHours	bonus
✓	10	-1	0	0	0	0	0	0
✓	10	0	-1	0	0	0	0	0
✓	10	0	0	-1	0	0	0	0
✓	10	0	0	0	-1	0	0	0
✓	10	0	0	0	0	-1	0	0
✓	10	0	0	0	0	0	-1	0
✓	10	0	0	0	0	0	0	-1

> ✓ classpath:features/payslip_formatting.feature

> ✓ classpath:features/search_employee.feature